



USER PRO · TIZADA PRO

Conectar otro sistema con TIZADA PRO

Guía completa para el programador: cómo se pone en marcha en el servidor, qué archivos manda y recibe el otro sistema, cómo se conecta cada JSON y cómo queda cada archivo de producción asociado a SU venta — sin que nadie toque TIZADA.

 Puesta en marcha

 Qué se manda

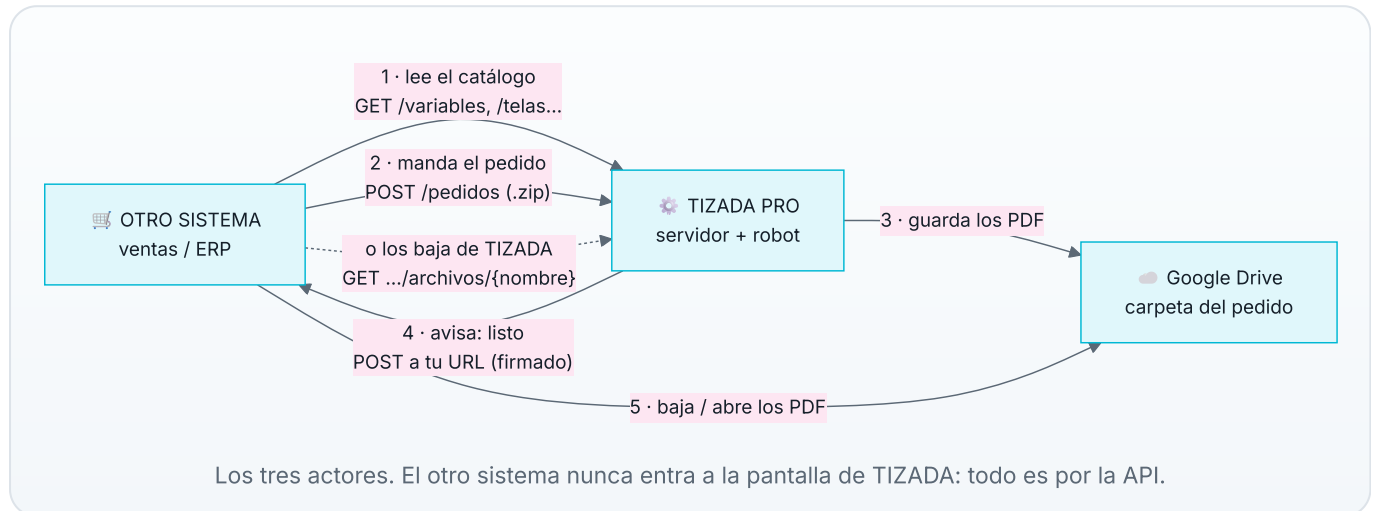
 Qué se recibe

 Cómo se conectan los JSON

 Aviso firmado

En una página

El otro sistema (ventas, ERP, tienda) **elige del catálogo de TIZADA** lo que se puede fabricar, **manda un paquete .zip** con el pedido y los artes, y **recibe los PDF de producción** —una tizada por tela y la ficha técnica— con la marca de SU venta. En el medio, un **robot** de TIZADA hace todo lo que haría una persona.



1 · Puesta en marcha

Una sola vez, en el servidor de TIZADA: actualizar, Node, la llave, el aviso y Google Drive.
Sección 1.



2 · Qué viaja

Lo que manda y lo que recibe el otro sistema, y cómo se conecta cada JSON con el anterior. **Secciones 2 a 5.**



3 · Probar y usar

Código listo para usar, lista de control y problemas frecuentes. **Secciones 6 a 8.**

1 Puesta en marcha en el servidor de TIZADA

2 El recorrido de un pedido

3 Qué se manda y qué se recibe

4 Cómo se conectan los JSON

5 El pedido, el resultado y el aviso, campo por campo

6 Estados y alarmas

7 Todas las rutas

8 Probarlo, lista de control y problemas frecuentes

📁 Los archivos que acompañan esta guía

Repositorio privado github.com/UserBreus/tizada-pro-integracion, carpeta `integracion/`: `API.md` (la API entera), `FORMATO_PEDIDO.md`, `openapi.yaml` (para Postman), `esquemas/` (JSON Schema), `pedido_ejemplo.json`, `resultado_ejemplo.json`, `respuestas/` (la respuesta REAL de cada ruta) y `codigo/` (Python y Node listos). El mismo repositorio trae un **sistema de ventas de prueba** que hace todo esto con pantalla.

Puesta en marcha

Lo hace quien administra el servidor de TIZADA PRO. Son siete pasos; al final, una prueba que dice si quedó bien.

1 Actualizar TIZADA PRO a una versión con la integración

La integración es del **2 de octubre de 2026** (versión 1.0.48 o posterior). Si el servidor tiene una copia anterior, primero va la actualización (el ZIP de actualización o `ACTUALIZAR-SISTEMA.bat` si la copia es de git). Se nota que está: en **Configuración** aparece la tarjeta **«Integraciones»**, y `GET /api/externo/v1/alarmas` contesta.

2 Instalar Node.js 20 o más nuevo

El **robot** que arma los pedidos que llegan es un programa de Node (`frontend/src/motor/robot/robot.mjs`) y lo arranca solo el servidor de TIZADA al iniciar. Sin Node, los pedidos se reciben pero quedan en cola. Lo que dice el robot queda en `logs/robot.log`.

3 Reiniciar TIZADA y mirar «El robot»

Configuración > Integraciones > El robot tiene que decir *«Prendido, esperando pedidos»* (punto verde). Si dice «No responde», el motivo está ahí mismo (por ejemplo, «no está instalado Node.js»).

4 Crear la llave del otro sistema

Integraciones > Llaves del otro sistema: escribir para quién es (p. ej. «Sistema de ventas») y tocar **«Crear llave»**. Se muestra **una sola vez**: «Copiar» y pasársela al programador del otro sistema por un canal seguro. TIZADA guarda sólo su huella. Para cortar el acceso: **«Anular»**.

5 El aviso al otro sistema (opcional, recomendado)

Integraciones > Aviso al otro sistema: la dirección donde el otro sistema recibe el aviso, p. ej. `https://ventas.ejemplo.com/tizada/aviso` → «Guardar». TIZADA le hace POST cuando cada pedido termina. Sin aviso, el otro sistema pregunta el estado cada pocos segundos.

6 Google Drive (para que los PDF queden en Drive)

1. En **Google Cloud** (proyecto propio): habilitar la **API de Google Drive** y crear un **ID de cliente OAuth** de tipo **«Aplicación web»**. En «URI de redireccionamiento autorizados» va la dirección de vuelta que **muestra la pantalla de Integraciones** (termina en `/api/integracion/drive/vuelta`). Bajar el `.json` del cliente.
2. En TIZADA: **«Cargar archivo de Google»** (ese `.json`) → **«Conectar con Google»** → aceptar con la cuenta dueña de las carpetas.
3. Pegar el enlace de la **carpeta de las tizadas** y el de la **carpeta de las fichas técnicas** → **«Guardar las carpetas»** → **«Probar»** (escribe un archivo de prueba en las dos).

Sin Drive igual funciona: los PDF quedan en el servidor y el otro sistema los baja por la API (`descarga`).

7 Que el otro sistema llegue a TIZADA (red)

El servidor del otro sistema tiene que poder llamar a `https://<servidor-de-TIZADA>/api/externo/v1` (puerto abierto / proxy). Y si hay aviso, TIZADA tiene que poder llamar a la dirección del paso 5. En internet, siempre **HTTPS**: un proxy adelante (IIS, nginx) con el certificado, como explica `DESPLIEGUE.md` §8. Si TIZADA vive bajo un subcamino (`https://.../Tizadapro`), la API queda en `https://.../Tizadapro/api/externo/v1`.

⚠ **Google: la app en «Prueba» vence a los 7 días**

Mientras la app de Google Cloud esté en modo «Prueba» con usuarios externos, el permiso de Drive se corta a los **7 días** y hay que volver a tocar «Conectar con Google». Para que no venza: **publicar la app** en Google Cloud (pide una página principal y una política de privacidad públicas) o usar una cuenta de **Google Workspace** con la app «Interna».

✅ **La prueba de que quedó bien**

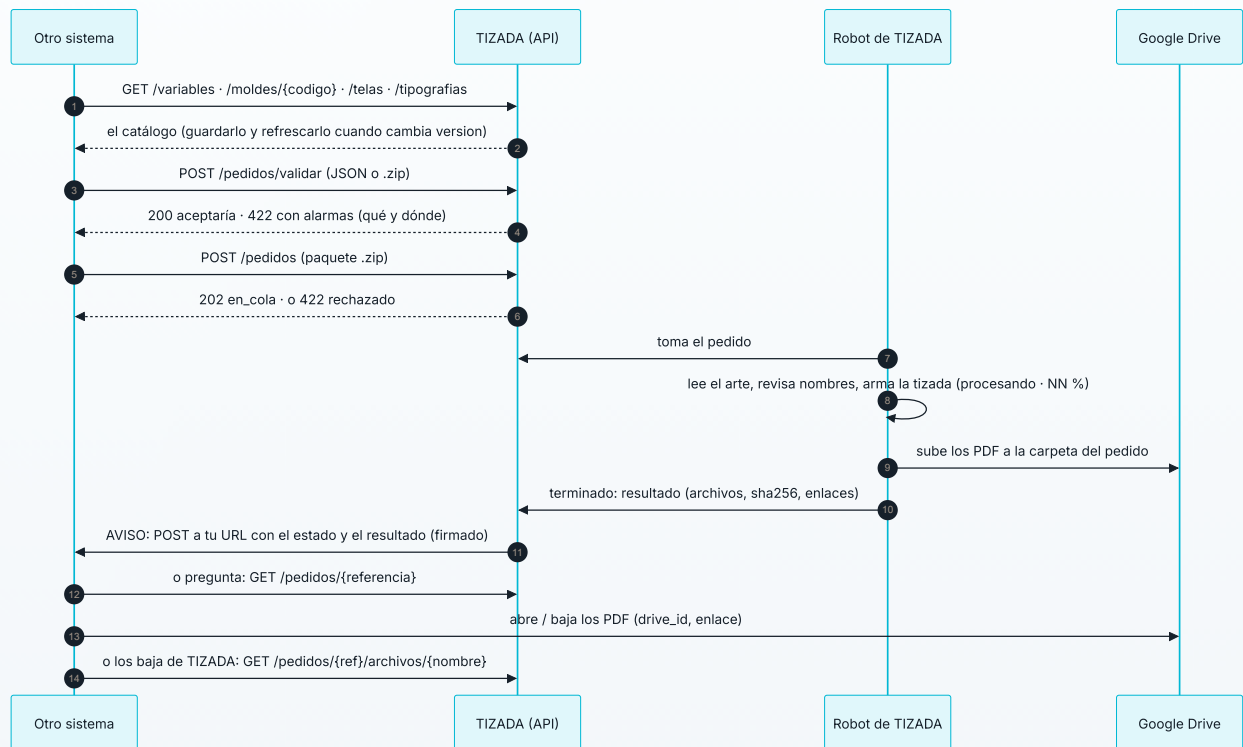
Desde la PC del otro sistema, con la llave: `python codigo/enviar_pedido.py una_carpeta_con_pedido --solo-revisar` → tiene que decir **«Revisión: la aceptaría (200)»**. Sin `--solo-revisar` manda el pedido, espera y baja los PDF comprobando cada sha256.

📁 **Dónde guarda TIZADA lo de la integración**

```
datos/externo/
├─ config.json          llaves (sólo la huella), aviso, Drive, robot
├─ pedidos/<referencia>/ paquete.zip tal cual llegó, pedido.json, estado.json, resultado.json
├─ salida/<referencia>/  copia de los PDF (siempre, haya Drive o no)
└─ rechazos.json        lo que se rechazó al llegar (se ve en la pantalla)
```

A los **30 días** de terminado, TIZADA borra el paquete y la **copia de los PDF del servidor** (quedan el estado y el resultado como historial; Drive no se toca). Desde ahí `descarga` contesta 404: el otro sistema baja los PDF apenas termina el pedido, o usa Drive.

El recorrido de un pedido



Lo que hace el otro sistema está a la izquierda; todo lo demás lo hace TIZADA sola.

enCola
esperando al robot



procesando
«etapa» dice en qué va
(y el %)



listo
trae el resultado

**rechazado · error ·
cancelado**
con las alarmas

Qué se manda y qué se recibe

Dirección	Qué	Formato	Ruta
→ TIZADA	El pedido : <code>pedido.json</code> + los artes + las tipografías que falten	un .zip	POST <code>/pedidos</code>
→ TIZADA	Revisar el pedido antes de mandarlo	el .zip o sólo el JSON	POST <code>/pedidos/validar</code>
← TIZADA	El catálogo : variables, prendas, talles, columnas de la planilla, telas, tipografías, alarmas	JSON	GET <code>/variables</code> y otras
← TIZADA	La silueta de cada variable (para mostrarla)	JSON con contornos SVG	GET <code>/variables/{clave}/foto</code>
← TIZADA	El estado y el resultado	JSON	GET <code>/pedidos/{ref}</code>
← TIZADA	El aviso cuando termina	JSON firmado (POST a tu URL)	webhook
← TIZADA	Los PDF : una tizada por tela (cada página es una mesa) + la ficha técnica A4	PDF	GET <code>.../archivos/{nombre}</code> o Drive

El paquete que se manda

```

0V-2026-00123.zip
├─ pedido.json           los datos (sección 5) — en la raíz
├─ artes/jugador_camiseta.ai   .ai (con «compatible con PDF») o .pdf
├─ artes/golero_camiseta.ai
└─ tipografias/Anton-Regular.ttf sólo las que TIZADA no tiene (.ttf / .otf)
  
```

Las rutas de adentro del zip son las que nombra `pedido.json`. Hasta **600 MB** y **400 archivos**. Se manda como archivo de formulario `paquete` (multipart) o como cuerpo `application/zip`.

Cómo tiene que venir el arte

El archivo

Una **mesa de trabajo por pieza**, armado sobre la base que da TIZADA para esa prenda. El dibujo **sin texto vivo** (convertido a curvas).

Las capas

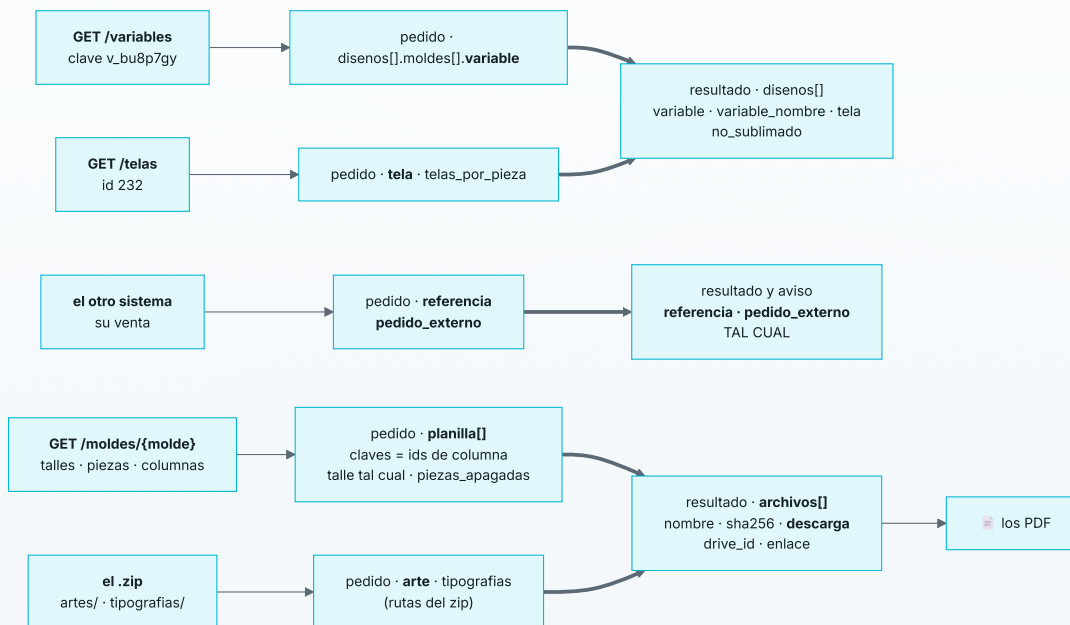
`diseño` (el dibujo) · `guias` (el nombre de la pieza de cada mesa) · `Nombre` y `Número` (lo que cambia en cada prenda) · una capa `Editable <objeto>` por cada objeto que puede ir aparte (TPU, DTF, bordado).

Lo que sale

Por cada pedido: **un PDF por tela** — `0V-2026-00123__H0JA_g0_<tela>.pdf`, cada página es una mesa lista para el RIP — y la **ficha técnica** `0V-2026-00123__FICHA_TECNICA.pdf` (tabla de talles y qué aplicar dónde). En Drive: las tizadas y `0V-2026-00123__resultado.json` en la carpeta del pedido, la ficha en la carpeta de fichas.

Cómo se conectan los JSON

Cada valor del pedido sale de algo que publica TIZADA, y cada cosa del resultado vuelve atada a lo que mandó el otro sistema. Ésta es la cadena:



Flechas finas: «de dónde sale el valor». Flechas gruesas: «vuelve atado a».

En el pedido	Sale de	Cómo va
disenos[].moldes[].variable	GET /variables → clave	La clave <code>v_...</code> . Una variable por prenda en cada diseño
...tela, ...telas_por_pieza (valores)	GET /telas → id (y las que admite la prenda: telas del molde)	El id , como texto: "232"
telas_por_pieza (claves), piezas_apagadas	GET /moldes/{molde} → piezas	Nombre genérico : «Cuello», no «Cuello 2»
Claves de cada fila de planilla[]	planilla.columnas[].id del molde	talle, nombre, numero, manga, cantidad ...
Valor de la columna de talla	talles[] del molde (y planilla.columna_talle)	Tal cual : "M", "2XL", "10"
Valores de columnas con lista (p. ej. manga)	columnas[].opciones y opciones_de_pieza	Una opción, o varias: "Corta + Larga"
planilla[].diseno	el nombre de uno de los disenos[] del mismo pedido	Obligatoria si hay más de un diseño
arte, tipografias	las rutas de adentro del .zip	"artes/jugador.ai"
editables (claves)	las capas Editable X del arte	X → sublimado · tpu · dtf · bordado
referencia, pedido_externo	el otro sistema	Vuelven tal cual en el resultado y el aviso

El pedido, el resultado y el aviso

pedido.json

```
{
  "formato": "tizadapro.pedido/1",
  "referencia": "OV-2026-00123",           ← única · carpeta en Drive y prefijo de cada PDF
  "pedido_externo": { "id": 48211, "sucursal": "Montevideo" }, ← libre: vuelve TAL CUAL
  "cliente": "Club Atlético Ejemplo",
  "opciones": { "si_piezas_en_blanco": "rechazar", "carpeta": "2026-10/OV-2026-00123" },
  "mesas": { "modo": "normal" },           ← o "una_por_fila", o "por_talles" + grupos
  "diseños": [
    { "nombre": "JUGADOR",
      "moldes": [ {
        "variable": "v_bu8p7gy",           ← cada entrada = una VARIABLE elegida
        "arte": "artes/jugador_camiseta.ai", ← GET /variables (Cuello redondo)
        "tela": "232", "telas_por_pieza": { "Cuello": "486" }, ← ids de GET /telas
        "editables": { "escudo": "sublimado", "logo": "tpu" } } ] },
    { "nombre": "GOLERO", "arte": "artes/golero_camiseta.ai",
      "moldes": [ { "variable": "v_t1mhcmu", "tela": "492",
        "editables": { "escudo": { "proceso": "bordado", "cruz": true } } } ] } ],
  "planilla": [
    { "diseno": "JUGADOR", "talle": "M", "nombre": "PÉREZ", "numero": "10", "manga": "Corta", "cantidad": 1 },
    { "diseno": "GOLERO", "talle": "L", "nombre": "GÓMEZ", "numero": "1", "manga": "Larga" } ]
}
```

Lo que vuelve: GET /pedidos/{referencia} (y el cuerpo del aviso)

```
{ "referencia": "OV-2026-00123", "estado": "listo", "etapa": "terminado",
  "pedido_externo": { "id": 48211, "sucursal": "Montevideo" }, ← el de la venta, tal cual
  "resultado": {
    "formato": "tizadapro.resultado/1", "tizada_id": 912,
    "destino": { "tipo": "drive", "carpeta_id": "1mJL5...", "carpeta_enlace":
      "https://drive.google.com/drive/folders/..." },
    "archivos": [
      { "tipo": "tizada", "nombre": "OV-2026-00123__HOJA_g0_Jacquard_Charr_a__1_83_.pdf",
        "bytes": 1127490, "sha256": "9c1e...", ← comprobar al bajarlo
        "descarga": "/api/externo/v1/pedidos/OV-2026-00123/archivos/...pdf",
        "drive_id": "1AbC...", "enlace": "https://drive.google.com/file/d/.../view",
        "tela": "Jacquard Charrúa (1,83)", "mesas": 1, "consumo_cm": 321, "aprovechamiento": 74.6 },
      { "tipo": "ficha", "nombre": "OV-2026-00123__FICHA_TECNICA.pdf", ... } ],
    "diseños": [ { "nombre": "GOLERO", "moldes": [ { "variable": "v_t1mhcmu", "variable_nombre": "cuello V",
      "no_sublimado": { "escudo": "bordado" } } ] } ],
    "alarmas": [ avisos que quedaron (no frenan) ] } }
```

El aviso y su firma

Lo que llega

POST a la dirección del aviso, con el mismo JSON de arriba y los encabezados X-Tizada-Referencia y X-Tizada-Firma. Se manda cuando el pedido queda **listo**, **rechazado** o **error**. Contestar **2xx**; si no, reintenta a los 5 s, 30 s y 2 min.

Cómo se comprueba

```
firma = HMAC-SHA256(clave = sha256_hex(llave), mensaje = cuerpo crudo)
en hexadecimal. Si no coincide, ignorarlo. Código listo: codigo/recibir_aviso.py y
firmaValida() en codigo/cliente.mjs.
```

Estados y alarmas

estado	Qué significa	Qué hace el otro sistema
en_cola	Recibido, esperando al robot	Esperar
procesando	Haciéndose; <code>etapa</code> es texto para mostrar, con <code>(NN %)</code> si se sabe el avance	Mostrar la etapa
listo	Terminó: viene <code>resultado</code>	Guardar los archivos (sha256, descarga, drive_id, enlace) en la venta
rechazado	El arte, las tipografías o el plan tienen algo que frena. No salió nada	Mostrar las alarmas, corregir y mandar de nuevo
error	Falla del sistema después de 3 intentos	Avisar a quien administra TIZADA
cancelado	Lo canceló el otro sistema (<code>DELETE</code>) o una persona	—

Cada alarma: `{ codigo, frena, etapa, mensaje, donde: { campo, fila } }`. **frena: true** rechaza; **false** es un aviso. `donde.campo` dice exactamente dónde (`disenos[1].moldes[0].tela` , `planilla[2].talle`). Son **67**, con qué significan y qué hacer: `GET /alarmas` (no pide llave).



Para no caer en...	Revisar antes contra...
<code>variable-desconocida</code> · <code>variable-ambigua</code>	<code>GET /variables</code> (mandar la clave <code>v_...</code>)
<code>molde-repetido</code>	Una variable por prenda en cada diseño
<code>tela-desconocida</code> · <code>tela-no-usable</code> · <code>tela-no-permitida</code>	<code>GET /telas</code> (<code>usable</code>) y las <code>telas</code> del molde
<code>talle-inexistente</code>	<code>talles[]</code> del molde
<code>opcion-inexistente</code> · <code>opcion-sin-piezas</code>	<code>columnas[].opciones</code> · <code>opciones_de_pieza</code>
<code>columna-obligatoria-vacia</code>	<code>columnas[].obligatoria</code>
<code>pieza-desconocida</code>	<code>piezas[]</code> del molde
<code>referencia-ya-generada</code>	La misma referencia ya está lista: <code>"reemplazar": true</code> para rehacerla
<code>piezas-en-blanco</code> · <code>arte-pieza-sin-mesa</code> · <code>editable-desconocido</code> · <code>tipografia-falta</code>	El arte (etapa arte): una mesa por pieza, las capas Editable y las tipografías del paquete

Todas las rutas

Base: `https://<servidor-de-TIZADA>/api/externo/v1` · encabezado `X-API-Key: tzp_...` · todo en UTF-8 · el detalle de cada una en `API.md` y `openapi.yaml`.

Método y ruta	Para qué	Contesta
GET <code>/variables</code>	Lo que se elige: las variables de cada prenda lista	200
GET <code>/variables/{clave}/foto</code>	Silueta vectorial de sus piezas	200 · 404 (<code>sin_calcular</code>)
GET <code>/moldes</code> · <code>/moldes/{codigo}</code>	Las prendas · todo lo de una (talles, piezas, planilla, opciones, telas)	200 · 404
GET <code>/moldes/{codigo}/foto</code>	Silueta de la prenda entera	200 · 404
GET <code>/telas</code> · <code>/tipografias</code>	Telas por id · tipografías que ya tiene	200
GET <code>/alarmas</code> · <code>/formato</code>	Catálogo de alarmas · formatos, límites, ejemplos (sin llave)	200
POST <code>/pedidos/validar</code>	Revisar sin guardar (JSON o .zip)	200 · 422
POST <code>/pedidos</code>	Mandar el pedido (.zip)	202 · 422 · 415
GET <code>/pedidos/{ref}</code>	Estado y resultado	200 · 404
GET <code>/pedidos/{ref}/archivos/{nombre}</code>	Bajar un PDF (<code>archivos[].descarga</code>)	200 (PDF) · 404
DELETE <code>/pedidos/{ref}</code>	Cancelar si todavía no se generó	200 · 409 · 404
<i>Aviso (webhook)</i>	TIZADA → POST a tu URL, firmado	contestar 2xx

🔑 La llave

Sin llave o con una anulada: 401 `{"error": "falta la llave o no es válida (encabezado X-API-Key)"}`. La llave sólo abre `/api/externo/v1` y firma los avisos: guardarla en el **servidor** del otro sistema, nunca en un navegador ni en el código de una página.

Probarlo, lista de control y problemas frecuentes

Probarlo en 5 minutos (código del repositorio)

```
# Python 3.8+, sin instalar nada
set TIZADA_URL=https://tizada.ejemplo.com
set TIZADA_LLAVE=tzp...
python integracion/codigo/enviar_pedido.py mi_pedido/ --solo-revisar      ← sólo revisa
python integracion/codigo/enviar_pedido.py mi_pedido/                    ← manda, espera y baja los
PDF (sha256)                                                             ← recibe el aviso en
python integracion/codigo/recibir_aviso.py                                :8090/aviso

# Node 18+
node integracion/codigo/cliente.mjs revisar pedido.json
node integracion/codigo/cliente.mjs mandar OV-2026-00123.zip
node integracion/codigo/cliente.mjs estado OV-2026-00123
node integracion/codigo/cliente.mjs bajar OV-2026-00123
```

O el **sistema de ventas de prueba** del mismo repositorio (`py app.py` con `TIZADA_URL` y `TIZADA_LLAVE`): arma pedidos como en TIZADA, los manda, muestra el avance, recibe el aviso y abre los PDF. Es la referencia funcionando.

☰ Lista de control

- ☐ TIZADA actualizado, Node instalado, el robot «Prendido».
- ☐ Llave creada y guardada en el servidor del otro sistema.
- ☐ El artículo del otro sistema guarda la **clave de la variable** (`v_...`).
- ☐ Telas por **id**, piezas por nombre **genérico**, talles **tal cual**.
- ☐ referencia única; `pedido_externo` con lo que identifica la venta.
- ☐ `/pedidos/validar` antes de `/pedidos` .
- ☐ Del resultado se guarda sha256, descarga, `drive_id` y enlace.
- ☐ El aviso se cree sólo con la firma correcta.

🔍 Problemas frecuentes

401: falta `X-Api-Key` o la llave fue anulada.

Queda en «en cola»: el robot no corre (Integraciones > El robot · `logs/robot.log` · ¿Node instalado?).

«drive-sin-configurar»: falta conectar Drive; los PDF igual se bajan con `descarga` (durante 30 días).

Drive dejó de subir a la semana: la app de Google sigue en «Prueba» (sección 1).

No llega el aviso: TIZADA no llega a esa dirección (red/HTTPS), o la firma no coincide (otra llave).

«variable-ambigua»: mandar la clave `v_...` , no el nombre.

Silueta 404 «sin calcular»: nadie abrió esa prenda en TIZADA todavía; mostrar un ícono.

Documento generado desde el código del sistema (1.0.48) y probado contra un TIZADA PRO real el 2 de octubre de 2026: un pedido de ejemplo hizo el recorrido completo (revisar, mandar, esperar, bajar 3 tizadas y la ficha con su sha256) y la firma del aviso se comprobó en Python y en Node. Reemplaza las secciones 8 a 13 de «La base de datos y cómo conectar otro sistema» (24/09/2026), que eran una propuesta anterior.